

БРОНЮВАННЯ ПЕРЕГОВОРНИХ

UA-Skills · event2 · конкурс для junior-розробників

ПРИЗОВИЙ ФОНД, ПЕРШІ 5 МІСЦЬ

UA-SKILLS.COM

01 ЗАВДАННЯ

Зробити невеликий веб-застосунок для бронювання переговорних в офісі. Співробітник відкриває розклад кімнати, бачить зайняті слоти і бронює вільний час. Своє бронювання можна скасувати, чуже чіпати не можна.

02 ЦО МАЄ ПРАЦЮВАТИ

- 1 Вхід.** Реєстрація: ім'я, email, пароль; вхід і вихід. Після перезавантаження сторінки сесія зберігається.
Правила:
 - email унікальний, перевіряється без урахування регістру та зайвих пробілів по краях (`Ivan@x.com` і `ivan@x.com` це одна адреса);
 - ім'я непорожнє, відображається в розкладі як автор бронювання; унікальність не вимагається;
 - пароль від 8 до 72 символів, вимог до складу немає;
 - перевірки виконуються на сервері, помилки показуються зрозумілим повідомленням.
- 2 Кімнати.** 5-6 переговорних створюються сідом: назва, поверх, місткість. Окрема адмінка не потрібна. Робочі години в усіх кімнат однакові: 09:00-19:00 за часом офісу (`Europe/Kyiv`).
- 3 Розклад кімнати.** Сітка на тиждень (дні по горизонталі, час по вертикалі, слоти по 30 хвилин). Зайняті слоти видно всім: назва бронювання і хто забронював. Можна гортати тижні вперед і назад. Орієнтир вигляду: тижневий вид Google Calendar.
- 4 Часовий пояс користувача.** Увесь час в інтерфейсі відображається в часовому поясі користувача (визначається з браузера). Користувач з іншого поясу бачить ту саму сітку зі зсувом: слот 10:00-10:30 за Києвом для користувача в Берліні відображається як 09:00-09:30. Якщо пояс користувача відрізняється від офісного, інтерфейс показує це (наприклад, підпис із поясом офісу поруч із сіткою). Перевірки робочих годин при цьому виконуються за часом офісу.
- 5 Створення бронювання.** Користувач обирає кімнату, дату, час початку і кінця, вводить назву. Правила:
 - назва обов'язкова, від 1 до 100 символів;
 - час кратний 30 хвилинам;
 - тривалість від 30 хвилин до 4 годин;
 - лише в робочі години і лише в майбутньому;
 - бронювання не може перетинатися з наявним;
 - бронювання впритул не конфліктують: кінець одного може збігатися з початком іншого (10:00-11:00 і 11:00-12:00 це два коректні бронювання).

- 6 **Помилки.** Якщо бронювання створити не можна, користувач бачить зрозуміле повідомлення: слот зайнятий, поза робочими годинами, час у минулому. Перевірки виконуються на сервері, а не лише у формі.
- 7 **Скасування.** Своє бронювання можна скасувати. Чуже не можна ні через інтерфейс, ні прямим запитом до API.
- 8 **Мої бронювання.** Сторінка зі списком своїх бронювань, дві секції або вкладки:
 - майбутні: найближче зверху, у кожного кнопка скасування;
 - минулі: останнє зверху, з пагінацією або підвантаженням;
 - рядок показує дату, час, кімнату і назву бронювання; час, як і скрізь, у поясі користувача;
 - клік по рядку веде до сітки цієї кімнати на відповідний тиждень.

03 ІНТЕРФЕЙС

Очікується продуманий інтерфейс, а не «форма поверх бази»:

- єдиний стиль на всіх екранах: відступи, типографіка, кольори (підхід до стилів будь-який: Tailwind, CSS-модулі тощо);
- в екранів є стани завантаження, порожнечі та помилки: «немає бронювань», недоступний сервер і порожня сітка не виглядають як білий екран або вічний спінер;
- у формах помилки показуються біля полів, кнопка блокується на час запиту;
- у сітці виділені поточний день і час, свої бронювання візуально відрізняються від чужих;
- скасування бронювання підтверджується (діалог або можливість скасувати дію);
- верстка не ламається на різних ширинах екрана (повноцінний мобільний сценарій залишається бонусом).

04 ТЕХНІЧНІ ВИМОГИ

- **Мова:** TypeScript.
- **Фронтенд:** React або Next.js.
- **Бекенд:** NestJS, Express або API-роути Next.js, на вибір учасника.
- **База даних:** PostgreSQL, MySQL або SQLite, на вибір учасника.
- Сітка розкладу верстається самостійно (таблиця або CSS grid). Готові календарні компоненти на кшталт FullCalendar використовувати не можна: ця верстка і логіка частина завдання.
- Час зберігається у форматі UTC.
- Паролі зберігаються хешованими (bcrypt або argon2).
- Сіди: кімнати (наприклад «Акваріум», «Марс», «Гагарін»), два тестові користувачі (облікові дані в README), кілька демо-бронювань.
- Юніт-тести на логіку перетину інтервалів: впритул, часткове перекриття, повний збіг, сусідні дні. Запускаються командою `npm test`.
- Секрети і налаштування в env, у репозиторії лежить `.env.example`.

05 БУДЕ ПЛЮСОМ

- Docker compose, що піднімає все однією командою.

- Підтвердження email у dev-режимі: реальний SMTP не потрібен, посилання підтвердження виводиться в лог сервера; до підтвердження бронювати не можна.
- Щотижневі повторювані бронювання (наприклад «щовівторка, 8 повторень») зі скасуванням одного повторення або всієї серії.
- Захист від гонки: два користувачі одночасно натискають «забронювати» на один слот, у базі має опинитися рівно одне бронювання. Обране рішення описується в README.
- Сповіщення про кінець бронювання: якщо наступний слот у цій кімнаті зайнятий, автор поточного бронювання за N хвилин до його кінця отримує сповіщення в застосунку (дзвіночок або тост). Сповіщення приходить рівно один раз і не приходить, якщо будь-яке з двох бронювань скасоване. N задається через env (наприклад `NOTIFY_BEFORE_MINUTES`, за замовчуванням 10 хвилин).
- Інтеграційні тести API: створення і скасування бронювання, відмови валідації.
- Фільтр кімнат за місткістю.
- Повноцінний мобільний сценарій: сіткою зручно користуватися на телефоні.

06 ЦО ЗДАВАТИ

Git-репозиторій з осмисленою історією комітів (проект, вивантажений одним комітом, знижує оцінку) і README: як запустити проект, як застосувати сіди, облікові дані тестових користувачів, які бонусні пункти реалізовані. Досить пари коротких абзаців своїми словами про те, як влаштована перевірка перетинів і як зберігається час.

Проект має запускатися за README на чистій машині без читання вихідного коду.

07 ЯК ОЦІНЮВАТИМЕМО

КРИТЕРІЙ	БАГА
Працює за списком «Що має працювати»	40
Якість коду: читабельність, структура, відсутність сміття і мертвого коду	25
UI/UX: очікування розділу «Інтерфейс», ясність сітки, загальна акуратність	20
README та історія комітів	15

Бонусні пункти додають бали зверху. Учасник має бути готовий пояснити будь-яку частину свого коду: на дзвінку після перевірки можливі питання, чому написано саме так.

Дискваліфікація. Доведені випадки спроби обману ведуть до дискваліфікації без виставлення балів: задача чужого рішення під своїм іменем, участь із двох і більше акаунтів, змова між учасниками.